PROJECT INVENTORY MANAGEMENT

Project titles: PROJECT INVENTORY MANAGEMENT SYSTEM

Team ID: SWTID-2026-3317

Team leader: KEERTHANA.T

keerthanatkeerthana44@gmail.com

Team members: KIRUTHIKA.A kiru26112006@gmail.com

KAVIYA.K: kk7994349@gmail.com

NITHEESHA.S: nitheeshasj@gmail.com

INTRODUCTION

Inventory management is a crucial part of every business organization. It refers to the process of ordering, storing, tracking, and controlling the company's stock or goods. Inventory can include raw materials, work-in-progress items, and finished products that are ready for sale. Proper inventory management helps businesses maintain the right quantity of products at the right time, ensuring smooth operations and customer satisfaction.

In today's competitive business environment, managing inventory efficiently is very important. Poor inventory control can lead to problems such as stock shortages, overstocking, increased storage costs, and loss of sales. Therefore, organizations use inventory management systems to monitor stock levels, track product movement, and plan future orders. These systems help businesses reduce operational costs and improve productivity.

Inventory management also plays an important role in supply chain management. It ensures that materials are available when needed for production and that finished products are delivered to customers on time. Modern companies often use computerized inventory systems and software to maintain accurate records, generate reports, and make better business decisions.

The main goal of inventory management is to balance supply and demand while minimizing costs. It helps businesses maintain efficiency, prevent waste, and increase profitability. Effective inventory management also improves customer satisfaction by ensuring that products are always available when customers need them.

This project focuses on understanding the concept of inventory management, its importance in business organizations, and the methods used to manage inventory effectively. It also highlights how proper inventory control contributes to the overall success and growth of a company.

Top of Form

Bottom of Form

Source code

```
# Simple Inventory Management System


inventory = {}


def add_product():
    name = input("Enter product name: ")
    quantity = int(input("Enter quantity: "))
    price = float(input("Enter price: "))


    inventory[name] = {"quantity": quantity, "price": price}
    print("Product added successfully.\n")


def view_inventory():
    if not inventory:
        print("Inventory is empty.\n")
        return


    print("\nInventory List")
    print("-------------------------")
    for name, details in inventory.items():
        print(f"Product: {name}")
        print(f"Quantity: {details['quantity']}")
        print(f"Price: {details['price']}")
        print("-------------------------")


def update_product():
    name = input("Enter product name to update: ")
```

```
    if name in inventory:
        quantity = int(input("Enter new quantity: "))
        price = float(input("Enter new price: "))

        inventory[name]["quantity"] = quantity
        inventory[name]["price"] = price
        print("Product updated successfully.\n")
    else:
        print("Product not found.\n")

def delete_product():
    name = input("Enter product name to delete: ")

    if name in inventory:
        del inventory[name]
        print("Product deleted successfully.\n")
    else:
        print("Product not found.\n")

def menu():
    while True:
        print("\nInventory Management System")
        print("1. Add Product")
        print("2. View Inventory")
        print("3. Update Product")
        print("4. Delete Product")
        print("5. Exit")

        choice = input("Enter your choice: ")
```

```
    if choice == "1":
        add_product()
    elif choice == "2":
        view_inventory()
    elif choice == "3":
        update_product()
    elif choice == "4":
        delete_product()
    elif choice == "5":
        print("Exiting program.")
        break
    else:
        print("Invalid choice. Try again.\n")

menu()
```

Project overview

- The Inventory Management System is developed to help businesses manage their stock efficiently and accurately. It allows users to store and maintain details of products such as name, quantity, price, and supplier information. The system helps track available stock, update inventory records, and monitor sales and purchases. By using this system, businesses can reduce manual work, avoid calculation errors, and save time. It also helps in maintaining proper stock levels and preventing shortages or overstocking. The project aims to provide a simple and user-friendly solution for organizing and controlling inventory data in a systematic and efficient way.

Mongodb connect page

File Edit Selection View Go Run ... product inventory management

EXPLORER

PRODUCT INVENTORY MANAGEMENT
- Code
  - server
    - node_modules
    - src
    - .env
    - .gitignore
    - package-lock.json
    - package.json
  - package-lock.json
- package-lock.json

.env | package-lock.json Code | package-lock.json ...\server

Code > server > package-lock.json > ...

```
6    "packages": {
73     },
74     "node_modules/balanced-match": {
75       "version": "1.0.2",
76       "resolved": "https://registry.npmjs.org/balanced-match/-/balanced-match-1.0.2.tgz",
77       "integrity": "sha512-3oSeUO0TMV67hN1AmbXsK4yaqU7tjiHlbxRDZOpH0KW9+CeX4bRAaX0Anxt0tx2MrpRpWwQaPwIlISE
78       "dev": true,
79       "license": "MIT"
80     },
81     "node_modules/bcryptjs": {
82       "version": "3.0.3",
83       "resolved": "https://registry.npmjs.org/bcryptjs/-/bcryptjs-3.0.3.tgz",
84       "integrity": "sha512-GlF5wPWnSa/X5LKM1o0wz0suXIINz1iHRLvTS+sLyi7XPbe5ycmYI3DlZqVGZZtDgl4DmasFg7gOB3J
85       "license": "BSD-3-Clause",
86       "bin": {
87         "bcrypt": "bin/bcrypt"
88       }
89     },
90     "node_modules/binary-extensions": {
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\acer\Desktop\product inventory management\code\server> npm run dev
[nodemon] starting `node src/server.js`
[dotenv@17.3.1] injecting env (3) from .env -- tip: auth for agents: https://vestauth.com
(node:22124) [MODULE_TYPELESS_PACKAGE_JSON] Warning: Module type of file:///C:/Users/acer/Desktop/product%20inventory%20man
agement/Code/server/src/server.js is not specified and it doesn't parse as CommonJS.
Reparsing as ES module because module syntax was detected. This incurs a performance overhead.
To eliminate this warning, add "type": "module" to C:\Users\acer\Desktop\product inventory management\Code\server\package.j
son.
(Use `node --trace-warnings ...` to show where the warning was created)
Server is running on port 5000
MongoDB connected !!
```

powershell
cmd
powershell
powershell
cmd
node server

OUTLINE

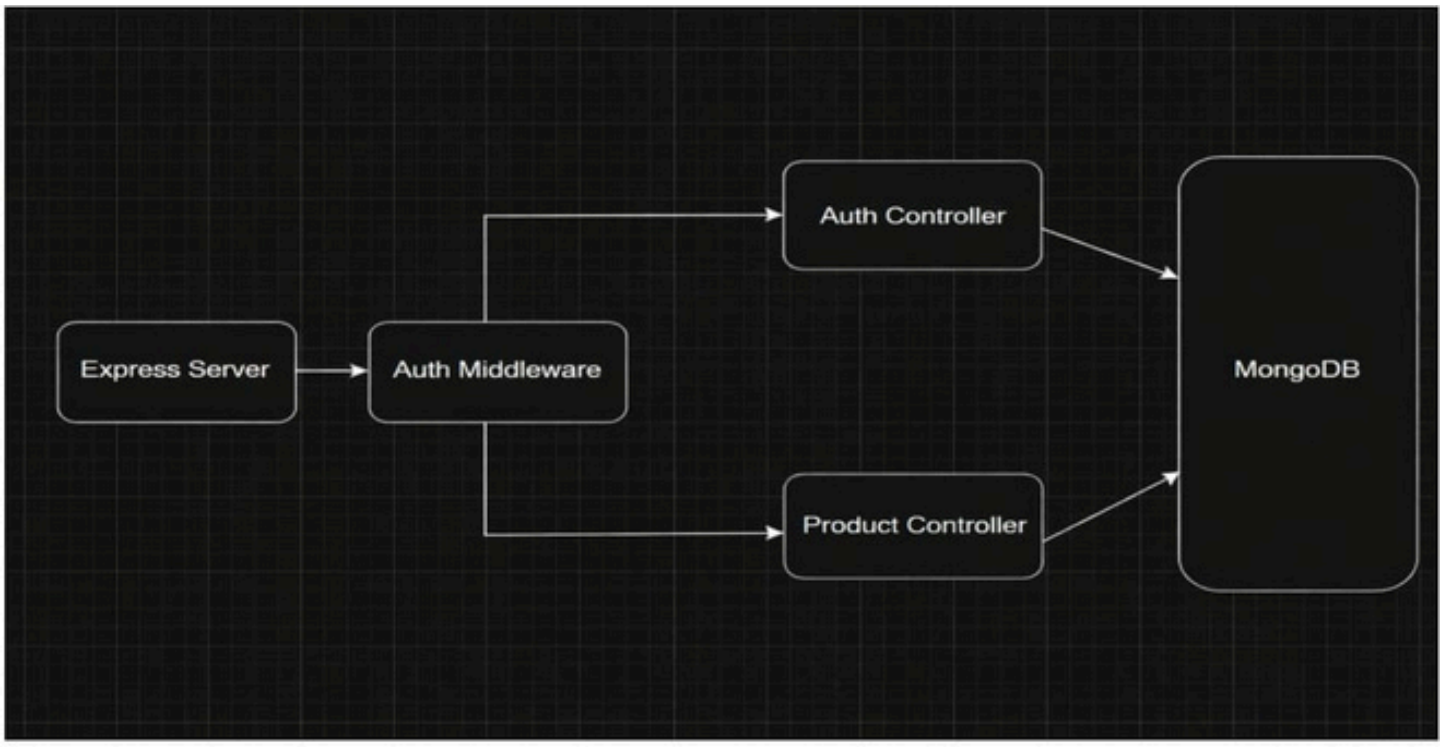
TIMELINE package-lock.json

Local History will track recent changes as you save them unless the file has been excluded or is too large. Source Control has not been configured.

Ln 1, Col 1 Spaces: 2 UTF-8 LF {} JSON

30°C Partly cloudy Search ENG IN 19:19 05-03-2026

Project Architecture



NODE.JS+EXPRESS BACKEND

- RESTful API architecture
- JWT authentication middleware
- MVC pattern(models,controllers,routes)
- INPUT validation and error handling

MongoDB database

- Document-based NOSQL storage
- Mongoose ODM for schema modeling
- Two collections: users and contacts
- Indexed fields for optimized queries

TECHNICAL ARCHITECTURE

Backend Architecture (Node.js + Express)

- **Controllers**: Handle authentication and product logic
- **Models**: Define database schema
- **Routes**: API endpoints for authentication and products
- **Middleware**: JWT verification and error handling
- **Configuration**: Environment variables and database connection

Database Architecture (MongoDB)

Users Collection:

- _id
- name
- email
- password (hashed)
- timestamps

Products Collection:

- _id
- userId (reference to user)
- name
- price
- quantity
- description
- timestamps

Relationship: One User → Many Products

Indexes:

- email (unique)
- userId
- product name

Backend development:

Setup Express Server:

Inside src/server.js, create the basic Express server.

Create controllers:

Inside: src/controllers/

Create:

auth.controller.js

Auth Controllers

The Auth Controller manages all user authentication-related operations in the system. It handles user registration and login functionality while ensuring secure access using JWT tokens.

Responsibilities:

- **Register new users**
- **Validate user input data**
- **Hash passwords using bcrypt before storing**
- **Authenticate users during login**
- **Generate JWT tokens for authorized access**
- **Send appropriate success and error responses**

Main Functionalities:

- **Register User: Creates a new user account after validating email uniqueness and encrypting the password.**
- **Login User: Verifies credentials and returns a JWT token for protected API access.**

Product controllers.js

Product Controller

The Product Controller handles all product inventory operations and contains the business logic for managing products linked to individual users.

Responsibilities:

- **Add new products to inventory**
- **Fetch products belonging to logged-in users**
- **Update product details like price, quantity, or description**
- **Delete products securely**
- **Perform product search functionality**

Main Functionalities:

- **Create Product: Adds a new product with a reference to the authenticated user.**
- **Get Products: Retrieves only the products created by the logged-in user.**
- **Update Product: Modifies product details while verifying ownership.**
- **Delete Product: Removes a product safely from the database.**
- **Search Product: Allows filtering products based on name.**

Create middeware:

Create :

Auth.routes.js

Create Models:

Inside src/models/ create:

- **User.js**
- **Product.js**

User model:

Product model:

Project Execution

Execution Steps

1. Open terminal in server folder.

2. Make sure you have a start script in the package.js file.

3. Run:

```
npm start
```

3. Console should display:

```
Server running on PORT
 Database Connected
```

ER Diagram:

ER diagram showing User → Product relationship:

Project Thumbnail:

Register user:

POST http://localhost:5000/api/v1/auth/register

Docs Params Authorization Headers (8) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "name": "User",
3   "email": "user@email.com",
4   "password": "123456"
5 }
```

Body Cookies Headers (10) Test Results 201 Created 428 ms 643 B

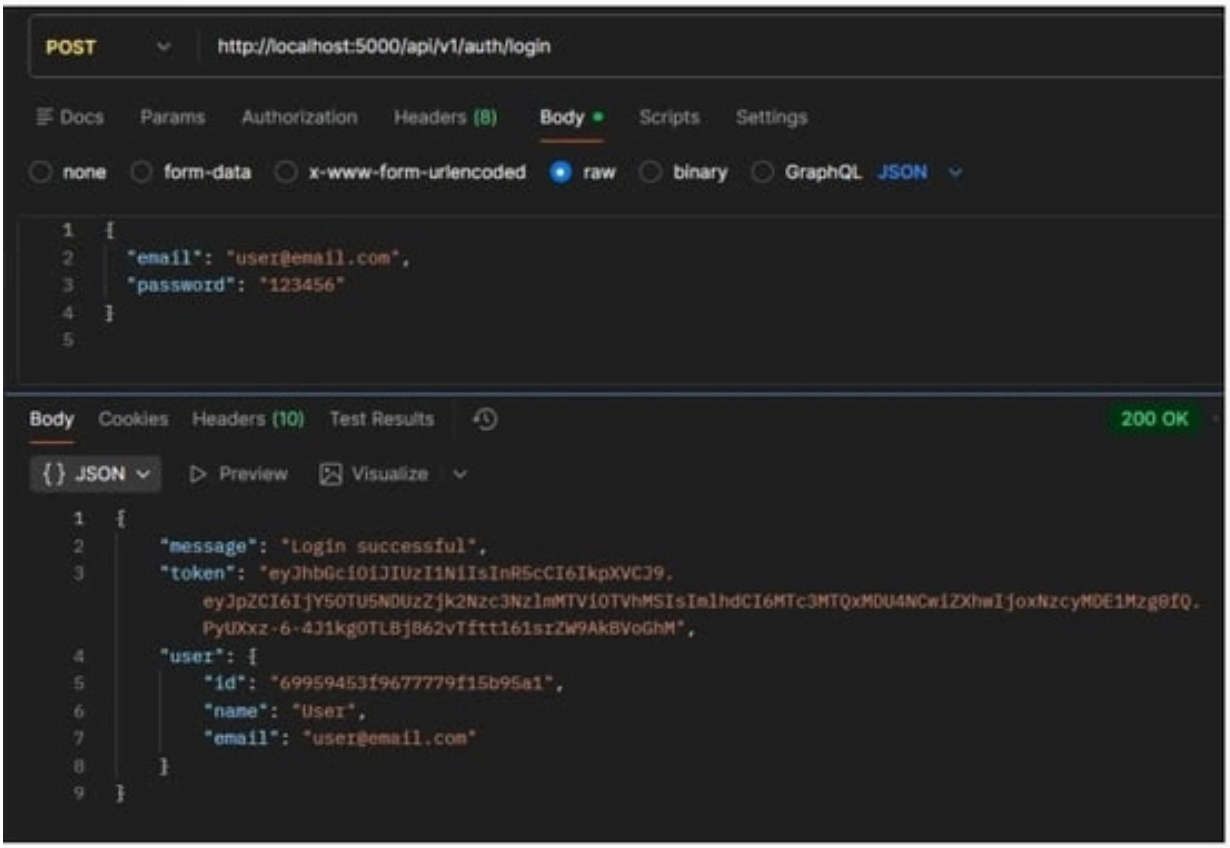
{} JSON Preview Visualize


```
1 {
2     "message": "Register successful",
3     "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
          eyJpZCI6IjY5OTU5NDUzZjk2Nzc3NzlmMTViOTVhMSIsImlhdCI6MTc3MTQxMDUxNSwiZXhwIjoxNzcyMDE1MzE1fQ.
          MyYzExe75OoxKIDXGhRjR_HrLvwfjaalt4LB4pgGBDE",
4     "user": {
5         "id": "69959453f9677779f15b95a1",
6         "name": "User",
7         "email": "user@email.com"
8     }
9 }
```


Register:

The register functionality allows new users to create an account by submitting their details ,which are validated are securely store with a hashed password.

Login user:

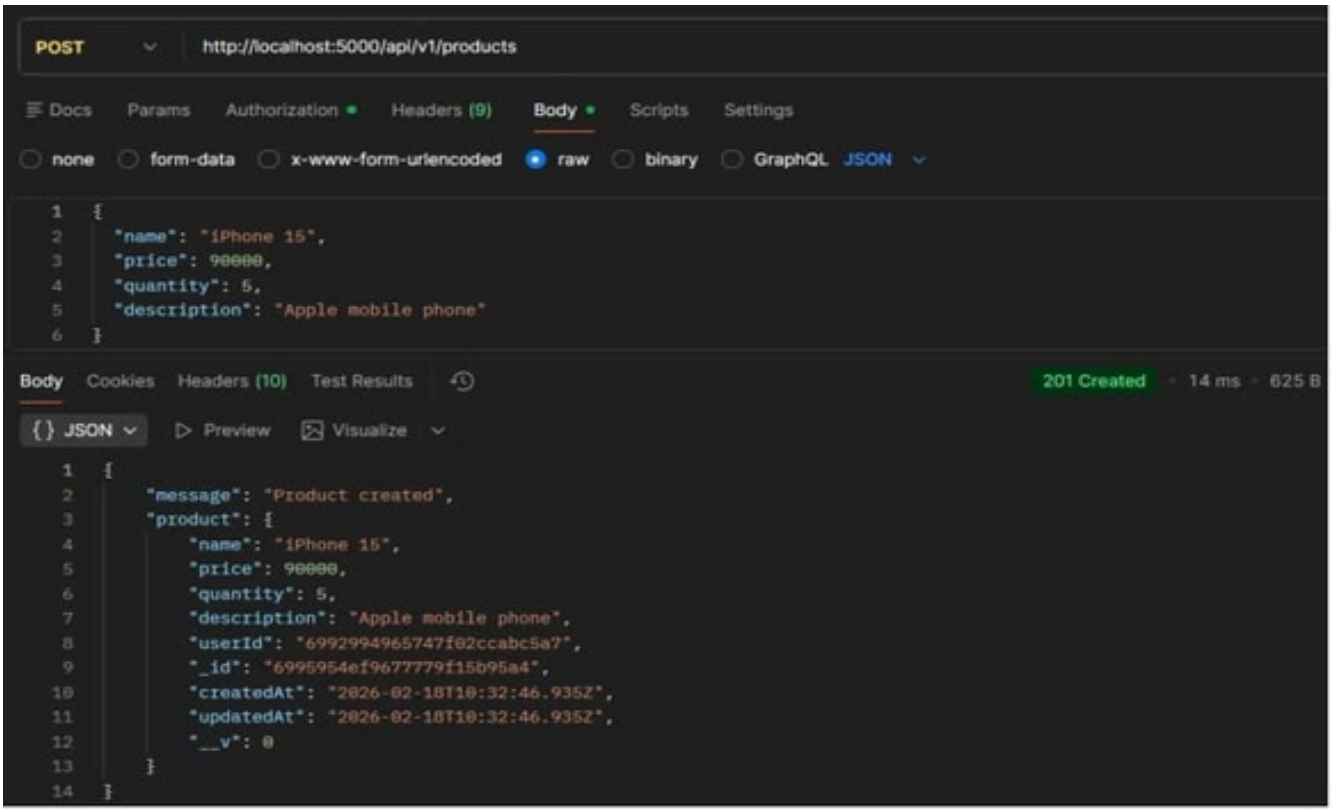


Login:

The login functionality verifies users credentials are generates a JWT token upon successful authentication .

This token is used access protected routes securely

Create product:



Get all product

Get all product

Fetches all products that belong to the logged-in user. It ensures users can view only their own inventory data.

Update product

Update product

Enables users to modify existing product details such as name, price, or quantity. The system verifies ownership

before allowing any updates.

Delete product

Delete product

Allows users to permanently remove a product from their inventory. The operation is protected to ensure only the product owner can delete it.

DEMO LINK :

https://drive.google.com/file/d/1nLdc5qfoYZUK8RTz2dwF-GNs2xjAY2Sc/view?usp=drivesdk